第4章 第5节 第一次API调用与核心参数

- 来源: https://class.imooc.com/lesson/5269#mid=104906
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: 小试牛刀，做一个简单的翻译器
- 课节: 图文 1-1 第一次API调用与核心参数

1-1 第一次API调用与核心参数

第4章 第5节 第一次API调用与核心参数

> [!TIP] 阅读指南 上一节我们完成了环境搭建。本节将跑通第一个API调用，并深入理解model、messages、role等核心参数——这些是后续所有开发的基础。

5.1 快速自测示例

新建你的项目目录，建议起一个正式的名字，因为我们后面还有很多地方需要写代码，最后集中在一个目录下管理。使用Qoder项目目录，在文件夹中创建一个名为test_qwen.py的文件，输入以下代码：

```
from openai import OpenAI

client = OpenAI(
    api_key="sk-qwen-xxxxxxxx",  # 你的Qwen密钥
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"  # Qwen地址
)

resp = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=[{"role": "user", "content": "你好，请简单介绍你自己"}]
)
print(resp.choices[0].message.content)

1
2
3
4
5
6
7
8
9
10
11
12

预览
```

> [!TIP] 完整可运行文件见本书资料：samples/chapter4/test_qwen.py

记得把sk-qwen-xxxxxxxx替换成你的真实密钥。

在Qoder的对话框中输入：

> 运行test_qwen.py

示例回复（可能略有差异）：

> 你好！我是Qwen，是阿里云研发的超大规模语言模型。我能够回答问题。我的训练数据截止时间是2024年12月，因此我对这一时间点之前的信息有较好的了解。

DeepSeek 快速自测示例

```
from openai import OpenAI

client = OpenAI(
    api_key="sk-deepseek-xxxxxxxx",  # 你的DeepSeek密钥
    base_url="https://api.deepseek.com"  # DeepSeek接口地址
)

resp = client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=[{"role": "user", "content": "你好，请简单介绍你自己"}]
)
print(resp.choices[0].message.content)
```

> [!TIP] 完整可运行文件见本书资料：samples/chapter4/test_deepseek.py

> 你好！很高兴认识你！ 我是DeepSeek，由深度求索公司创造的AI助手。让我简单介绍一下自己：我的特点 知识截止到2024年7月，是DeepSeek最新版本模型 ... 我能帮你做什么： ...

你可以看到无论是DeepSeek还是Qwen他们的代码几乎一模一样，只是配置需要更换一下。

这主要是因为他们都兼容OpenAI的API，使用的也是OpenAI的开发SDK。本文后续不再演示DeepSeek，主要还是使用Qwen模型。

此外，可以看到他们的回复都包含自己训练数据的截止日期。这些截止日期大多数早于当前时间，在学习了第1章的知识后，应该明白为什么是这样。

5.2 第一次调用：Hello AI

现在，开始写人生第一个AI程序。

代码：最简单的调用

> [!TIP] 完整源码参考：samples/chapter4/hello_ai.py

创建一个文件hello_ai.py，输入以下代码：

```
from openai import OpenAI

# 使用 system 角色并返回结构化结果
client = OpenAI(
    api_key="sk-xxxxxxxxxxxxxxxxxxx",  # 你的Qwen密钥
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"  # Qwen兼容接口地址
)

messages = [
    {
        "role": "system",
        "content": "你是一个专业的AI助手。请仅以JSON格式返回结果。字段：intro（简短自我介绍），capabilities（列出3项能力）。"
    },
    {
        "role": "user",
        "content": "请介绍你自己，并说明你能做什么。"
    }
]

response = client.chat.completions.create(
    model="qwen3.6-plus",  # 当前示例使用 Qwen3.6-Plus
    messages=messages
)

# 打印JSON结构的回复与用量信息
print(response.choices[0].message.content)
print("tokens:", response.usage.total_tokens)
```

保存文件后，有两种执行方式，任选其一即可：

传统方式

在终端中运行：

```
python3 hello_ai.py

1
```

AI Agent方式

> 运行hello_ai.py

Qoder会在内置终端中执行，并直接显示结果。

> [!NOTE] 后续章节中，除非代码执行方式有特殊之处，否则不再重复说明如何执行代码。读者可以自行选择使用传统方式或AI Agent方式。

几秒钟后，会看到AI的回复（仅为示例数据）：

```
{
  "intro": "我是一个专业的AI助手，致力于提供准确、高效和有用的信息与服务。",
  "capabilities": [
    "回答各种主题的问题，包括科学、技术、文化、历史等",
    "协助进行文本生成、编辑和总结",
    "提供逻辑推理、数学计算和编程支持"
  ]
}
```

恭喜！ 已经完成了第一次AI API调用。

@logicframe-video{19}

调用流程总览与逐行解析

先看整体流程（总览）：

虽然只有短短十几行代码，但每一行都很重要。下面逐行解读：

导入库

```
from openai import OpenAI

1
```

导入OpenAI的Python客户端库。虽然我们用的是Qwen，但因为兼容OpenAI接口，所以用同一个库。

初始化客户端

```
client = OpenAI(
    api_key="sk-qwen-xxxxxxxx",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
```

- api_key：你的API密钥，用于身份验证和计费
- base_url：API的服务器地址 Qwen的地址：https://dashscope.aliyuncs.com/compatible-mode/v1

发送请求

```
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=[
        {"role": "user", "content": "你好，请介绍一下你自己"}
    ]
)
```

- model：指定使用的模型 qwen3.6-plus：Qwen的对话模型（均衡型） 其他Qwen模型：可按需要选择
- messages：对话消息列表 role：角色，这里是user（用户） content：消息内容

获取回复

```
print(response.choices[0].message.content)


1
```

- response.choices：API返回的候选回复列表（通常只有1个）
- [0]：取第一个候选
- .message.content：提取回复的文本内容

5.3 关键参数说明

理解完整个代码流程后，需要对一些关键概念进行说明。

> [!TIP] API文档 Qwen 文档：https://help.aliyun.com/zh/model-studio/model-api-reference/
> 我们不会把文档内容复制过来，这里主要说明一些必须要了解的模型参数，帮助你快速理解基本概念，节约学习时间。

model参数：选择合适的模型

model参数指定使用哪个具体的AI模型。不同模型有不同的特点、能力和成本。

以通义千问为例，通义千问实际上是一个模型家族，可选模型多达上百种。除了文本生成模型，还包括音视频、图形图像、自主编程、数学等其他专有能力模型。这里我们聚焦常用的文本模型，按功能划分为以下模型系列，每个系列针对不同的使用场景：

- 通义千问3.6系列（2026年4月最新） 通义千问已经发布3.6系列，在编程Coding能力、智能体Agent能力和工具调用能力方面有显著提升： Qwen3.6-Max-Preview：能力最强，世界知识和指令遵循能力增强，适合复杂任务 Qwen3.6-Plus：效果、速度、成本均衡，重点增强Agentic Coding智能体编程能力，提升深度推理与多模态识别（OCR/万物识别） Qwen3.6-Flash：适合简单任务，速度快、成本低 Qwen3.6-27B：270亿参数稠密多模态模型，已开源，编程能力超越15倍规模MoE模型 适用场景：根据任务复杂度和预算选择，复杂推理用Max，日常任务和智能体用Plus，简单高频用Flash

示例：选择不同模型系列

```
# 场景1：需要推理比如Agent → 使用 plus系列
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=[{"role": "user", "content": "用Python实现快速排序"}]
)

# 场景2：简单翻译 → 使用 Flash系列
response = client.chat.completions.create(
    model="qwen3.6-flash",
    messages=[{"role": "user", "content": "翻译成英文：你好"}]
)

# 场景3：复杂任务
response = client.chat.completions.create(
    model="qwen3.6-max-preview",
    messages=[{"role": "user", "content": "证明：根号2是无理数"}]
)
```

能力越强的模型，消耗Token越多。

对话式API vs 传统API（非常重要）

在深入参数细节前，先要打破一个常见的思维定式：大模型的API是对话式API，它和传统API的"一次性请求"不太一样。

我们习惯了这样的API调用模式：发一次请求拿一次结果，上一次与下一次API调用彼此独立。

但大模型的对话式API完全不同——它需要历史信息来理解你的问题，所以上一次调用和下一次调用是有关联的。也就是说，下一次调用API时，你需要携带历史消息(如果有的话），这是对话式API和传统API最大的不同。

接下来先看两个对比示例代码，然后再用两张图帮助你建立直观认知。

传统API（一次性请求，固定选项参数）：

```
import requests

BASE_URL = "API_BASE_URL"  # 传统接口地址（示意）

# 第1次：获取今天的天气，不依赖历史
resp_today = requests.get(
    BASE_URL + "/weather/forecast",
    params={"city": "CITY_123", "day": "today"}
)

# 第2次：获取明天的天气，仍然不需要上一次的结果
resp_tomorrow = requests.get(
    BASE_URL + "/weather/forecast",
    params={"city": "CITY_123", "day": "tomorrow"}
)
```

AI对话式API（多轮交互，必需携带历史对话数据）：

```
# 伪代码：携带历史的对话式调用
history = []

# 第1轮：提问并假设返回的文本内容
resp1 = call_ai(question="我正在开发一个电商网站", history=history)
resp1_text = "很好！电商网站需要考虑商品管理、订单处理、支付集成等模块。你想从哪个部分开始？"  #

# 将上一轮的回答文本加入历史，便于下一轮继续追问
history.append(resp1_text)

# 第2轮：继续追问（这里的"它"指代第1轮提到的"电商网站"，必须依赖历史才能理解）
resp2 = call_ai(question="它需要用到什么数据库？", history=history)
resp2_text = "对于电商网站，推荐使用MySQL存储商品、订单等结构化数据，Redis做缓存层提升性能。"  #

# AI理解了"它"指代电商网站
# 同样把这轮的回答加入历史
history.append(resp2_text)

# 第3轮：在已有历史基础上继续深入（"这两个"指代上一轮提到的MySQL和Redis）
# 如果不携带历史，AI根本不知道"这两个"是什么
resp3 = call_ai(question="这两个怎么配合使用？", history=history)
```

下面用一张时序图串起两轮调用，直观理解"携带历史"的节奏：

> 每次请求都要带上"对话上下文历史信息"，这个历史信息就是 messages 参数。

下面就来拆解 messages 的具体结构。

messages参数的技术结构

回到前面的 Hello AI 示例，那段代码已经给出了 messages 的基本形态：

```
messages = [
    {
        "role": "system",
        "content": "你是一个专业的AI助手。请仅以JSON格式返回结果。字段：intro（简短自我介绍），capabilities（列出3项能力）。"
    },
    {
        "role": "user",
        "content": "请介绍你自己，并说明你能做什么。"
    }
]
```

这段代码包含了两条消息：一条是系统指令，设定了AI的角色和输出格式要求；另一条是用户的提问。它们被放在一个数组里，按时间顺序排列。

messages 数组就是承载这些历史对话的容器。每条消息都是一个字典，必须包含两个核心字段：

```
{
    "role": "角色类型",      # 必需：指定消息发送者（system/user/assistant）
    "content": "消息内容"    # 必需：实际的文本内容
}
```

为什么是数组?

> 因为对话是连续的时间序列。数组保证了历史消息按顺序传递，让模型理解"谁在什么时候说了什么"。

最小有效示例：

```
messages = [
    {"role": "user", "content": "什么是质数？"}
]
```

messages 至少要有1条消息，且每条必须包含 role 和 content 字段。

三轮对话演进示例：

假设用户问：“推荐一部科幻电影”，AI回答了《星际穿越》。用户接着问：“导演是谁？”，AI回答后用户继续问：“他还拍了什么？”。

整个过程中，messages 数组的演变如下：

- 第1轮 (用户提问)：

```
messages = [
    {"role": "user", "content": "推荐一部科幻电影"}
]
# 请求后，AI返回《星际穿越》
```

- 第2轮（用户追问"导演是谁"）：

```
messages = [
    {"role": "user", "content": "推荐一部科幻电影"},
    {"role": "assistant", "content": "推荐《星际穿越》（Interstellar），由克里斯托弗·诺兰执导..."},
    {"role": "user", "content": "导演是谁？"}
]
# AI能理解"导演"指的是《星际穿越》的导演，回答"克里斯托弗·诺兰"
```

- 第3轮（用户追问"他还拍了什么"）：

```
messages = [
    {"role": "user", "content": "推荐一部科幻电影"},
    {"role": "assistant", "content": "推荐《星际穿越》（Interstellar），由克里斯托弗·诺兰执导..."},
    {"role": "user", "content": "导演是谁？"},
    {"role": "assistant", "content": "导演是克里斯托弗·诺兰（Christopher Nolan）。"},
    {"role": "user", "content": "他还拍了什么？"}
]
# AI能理解"他"指代诺兰，回答《盗梦空间》《信条》等作品。
# 如果不携带历史，AI根本不知道"他"是谁。
```

通过这种历史积累，AI能在多轮对话中保持上下文连贯。每次请求时，都要把之前的对话完整传入。

role参数的三种取值

role，意为角色，这个参数表明当前消息的角色是什么，它有3个取值。

system（系统角色）

- 作用：设定AI的"人设"与行为规则

作用

- 特点：最高优先级，贯穿整个对话 ，在数组中只会出现一次

- 位置：通常放在 messages 数组的第一条

位置

- 是否必需：可选，但强烈建议使用

是否必需

强烈建议使用

典型用途：

- 定义AI的角色身份（“你是Python导师”）
- 约束输出格式（“只返回JSON格式”）
- 设定回答风格（“讲解要通俗易懂”）

> [!IMPORTANT] 为什么要设定AI角色？ 同样的问题，不同角色给出的回答深度完全不同。 例如问"什么是递归？"： 设定为"小学老师" → AI会用汉诺塔等简单比喻 设定为"算法专家" → AI会深入分析调用栈与时间复杂度

示例：

```
messages = [
    {
        "role": "system",
        "content": "你是一位Python编程导师，擅长用简单语言解释复杂概念。回答时要分步骤，配合代码示例。"
    },
    {
        "role": "user",
        "content": "什么是装饰器？"
    }
]
```

user（用户角色）

- 作用：表明这是用户的输入信息（问题）
- 位置：可以有多条（多轮对话）
- 是否必需：必需，至少要有一条

role最常见的取值，通常就是用来记录用户的问题。

```
messages = [
    {"role": "user", "content": "帮我写一个计算阶乘的函数"}
]
```

assistant（助手角色）

- 作用：表明这是AI的历史回复
- 位置：穿插在 user 消息之间
- 是否必需：单轮对话不需要，多轮对话必需

多轮对话必需

> [!IMPORTANT] 每次API调用都是独立的，模型不会自动记住历史。如果你需要连续对话，你应当手动将之前的 user 和 assistant 消息都传入 messages 数组。

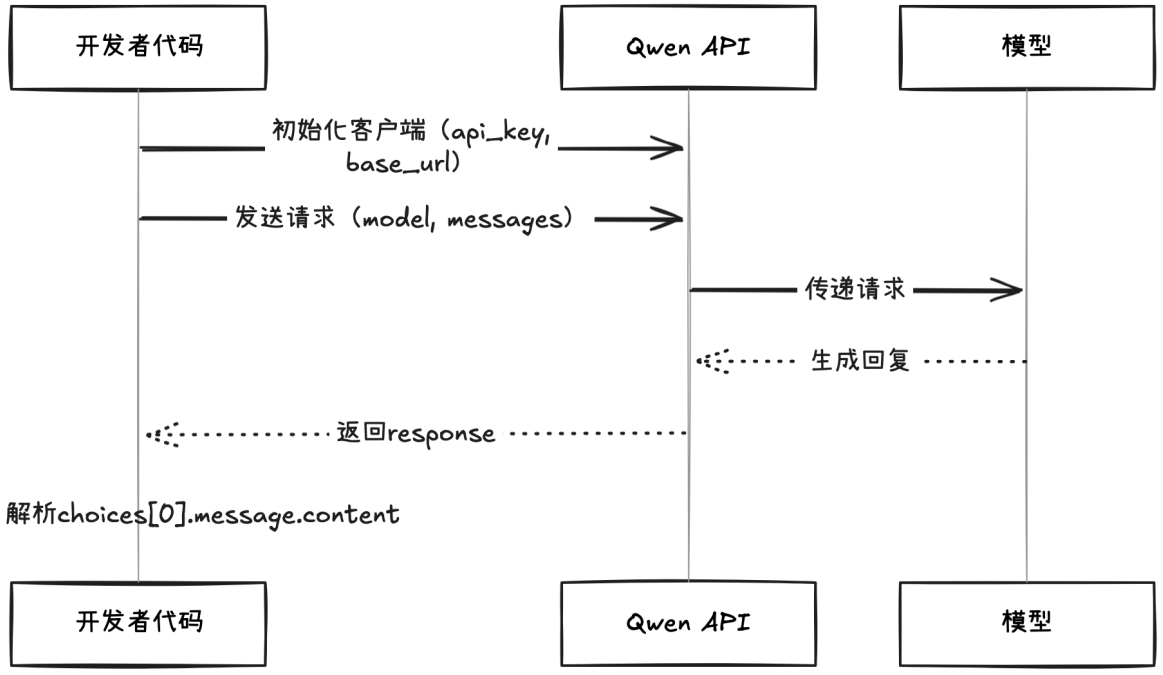
5.4 下一节预告

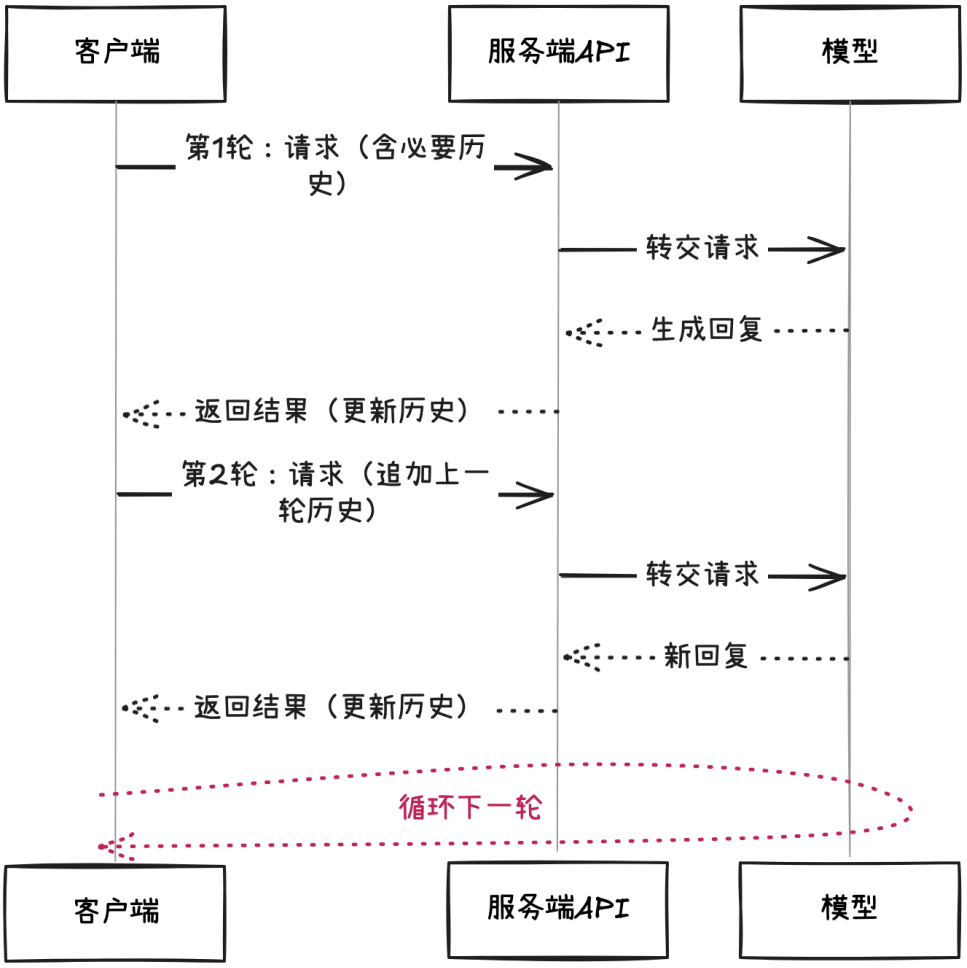
掌握了API调用的核心参数后，接下来进入实战阶段——多轮对话、上下文管理策略，最终实现一个智能翻译器。

5.5 ■ 学点英语

中文	English	音标	说明
API调用	API Call	/ˌeɪ.piːˈaɪ kɔːl/	通过编程接口向AI模型发送请求并获取回复
系统角色	System Role	/ˈsɪstəm roʊl/	设定AI人设与行为规则的messages角色
用户角色	User Role	/ˈjuːzər roʊl/	表示用户输入信息的messages角色
助手角色	Assistant Role	/əˈsɪstənt roʊl/	记录AI历史回复的messages角色
模型选择	Model Selection	/ˈmɑːdl sɪˈlekʃn/	根据任务复杂度选择不同能力级别的模型

图片


开发者代码
Qwen API
模型
初始化客户端（api_key, base_url）
发送请求（model, messages）
传递请求
生成回复
返回response
解析choices[0].message.content
开发者代码
Qwen API
模型


客户端
服务端API
模型
第1轮：请求（含必要历史）
转交请求
生成回复
返回结果（更新历史）
第2轮：请求（追加上一轮历史）
转交请求
新回复
返回结果（更新历史）
循环下一轮
客户端
服务端API
模型